

并行与分布式计算

-- 容错

Dependability可靠性

基本概念

组件向客户端提供服务。为了提供服务，组件可能需要其他组件的服务 $\Rightarrow$ 组件可能依赖于其他组件。

具体来说

如果组件 C 的行为的正确性取决于 C^* 的行为的正确性，则该组件 C 依赖于 C^* 。（组件是流程或渠道）

Dependability可靠性

基本概念

组件向客户端提供服务。为了提供服务，组件可能需要其他组件的服务⇒组件可能依赖于其他组件。

具体来说

如果组件 C 的行为的正确性取决于 C^* 的行为的正确性，则该组件 C 依赖于 C^* 。（组件是进程）

与可靠性相关的要求

Requirement	Description
Availability	Readiness for usage
Reliability	Continuity of service delivery
Safety	Very low probability of catastrophes
Maintainability	How easy can a failed system be repaired

可靠性与可用性

组件C的可靠性 $R(t)$

假设 C 在时间 $T = 0$ 时正常运行，则 C 在 $[0, t)$ 期间正常运行的条件概率。

典型指标

- ▶ **Mean Time To Failure** 平均故障时间 ($MTTF$): 组件发生故障的平均时间。
- ▶ **Mean Time To Repair** 平均修复时间 ($MTTR$): 修复组件所需的平均时间。
- ▶ **Mean Time Between Failures** 平均故障间隔时间 ($MTBF$): 简单来说就是 $MTTF + MTTR$ 。

可靠性与可用性

组件C的可用性 $A(t)$

C 在区间 $[0, t)$ 内启动并运行的平均时间比例。

► Long-term availability A 组件的长期可用性: $A(\infty)$

► **Note:** $A = \frac{MTTF}{MTBF} = \frac{MTTF}{MTTF+MTTR}$

观察

只有当我们准确了解故障的真正含义时，可靠性和可用性才有意义。

术语

Failure失败, error错误, fault故障

Term	Description	Example
Failure	A component is not living up to its specifications	Crashed program
Error	Part of a component that can lead to a failure	Programming bug
Fault	Cause of an error	Sloppy programmer

Terminology术语

Handling faults处理故障

Term	Description	Example
Fault prevention	Prevent the occurrence of a fault	Don't hire sloppy programmers
Fault tolerance	Build a component such that it can mask the occurrence of a fault	Build each component by two independent programmers
Fault removal	Reduce the presence, number, or seriousness of a fault	Get rid of sloppy programmers
Fault forecasting	Estimate current presence, future incidence, and consequences of faults	Estimate how a recruiter is doing when it comes to hiring sloppy programmers

故障模型

故障类型

Type	Description of server's behavior
Crash failure 崩溃故障	Halts, but is working correctly until it halts
Omission failure 遗漏故障 <i>Receive omission</i> <i>Send omission</i>	Fails to respond to incoming requests Fails to receive incoming messages Fails to send messages
Timing failure 时间故障	Response lies outside a specified time interval
Response failure 响应故障 <i>Value failure</i> <i>State-transition failure</i>	Response is incorrect The value of the response is wrong Deviates from the correct flow of control
Arbitrary failure 任意故障	May produce arbitrary responses at arbitrary times

可靠性与安全性

Omission versus commission疏忽与过失

任意故障有时被视为恶意故障。最好进行以下区分：

- ▶ 疏忽故障：组件未能采取其本应采取的行动
- ▶ 过失故障：组件采取了其本不应该采取的行动

可靠性与安全性

Omission versus commission疏忽与过失

任意故障有时被视为恶意故障。最好进行以下区分：

- ▶ 疏忽故障：组件未能采取其本应采取的行动
- ▶ 过失故障：组件采取了其本不应该采取的行动

观察

请注意，故意故障（无论是疏忽还是故意故障）通常都是安全问题。通常，区分故意故障和无意故障是不可能的。

Halting failures 停机故障

场景

C 不再感知到来自 C^* 的任何活动 - 停机故障？区分崩溃或遗漏/计时故障可能是不可能的。

异步系统与同步系统

- ▶ 异步系统：不假设进程执行速度或消息传递时间 → 无法可靠地检测崩溃故障。
- ▶ 同步系统：进程执行速度和消息传递时间有界 → 我们可以可靠地检测遗漏和时间故障。
- ▶ 实际上，我们有部分同步系统：大多数时候，我们可以假设系统是同步的，但系统异步的时间没有限制 → 通常可以可靠地检测崩溃故障。

Halting failures停机故障

假设

Halting type	Description
Fail-stop	Crash failures, but reliably detectable
Fail-noisy	Crash failures, eventually reliably detectable
Fail-silent	Omission or crash failures: clients cannot tell what went wrong
Fail-safe	Arbitrary, yet benign failures (i.e., they cannot do any harm)
Fail-arbitrary	Arbitrary, with malicious failures

通过冗余掩盖故障

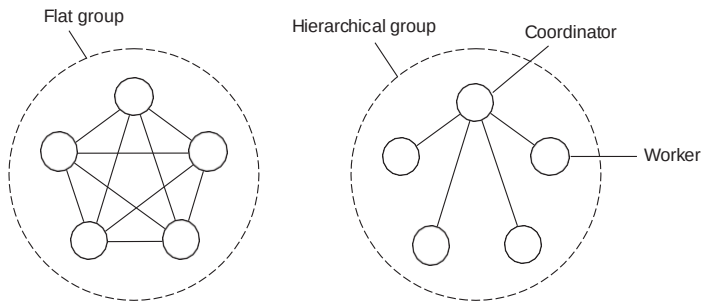
冗余类型

- ▶ **Information redundancy**信息冗余: 向数据单元添加额外的位, 以便在位混乱时可以恢复错误。
- ▶ **Time redundancy**时间冗余: 设计一个系统, 以便如果出现任何问题, 可以再次执行操作。通常用于故障是暂时的或间歇性的。
- ▶ **Physical redundancy**物理冗余: 添加设备或进程, 以允许一个或多个组件发生故障。这种类型在分布式系统中广泛使用。

按进程组进行复原

基本思想

通过进程复制来防止进程发生故障，将多个进程组织成一个进程组。
区分平面组和层级组。



Consensus共识

先决条件

在容错进程组中，每个无故障进程都按照与其他无故障进程相同的顺序执行相同的命令。

重构

无故障组成员需要就下一步执行哪个命令达成共识。

Flooding-based consensus基于洪泛的共识

系统模型

- ▶ 进程组 $\mathbf{P} = \{P_1, \dots, P_n\}$
- ▶ **Fail-stop** 故障语义，即具有可靠的故障检测
- ▶ 客户端联系 P_i 请求其执行命令
- ▶ 每个 P_i 都维护一个建议命令列表

Flooding-based consensus基于洪泛的共识

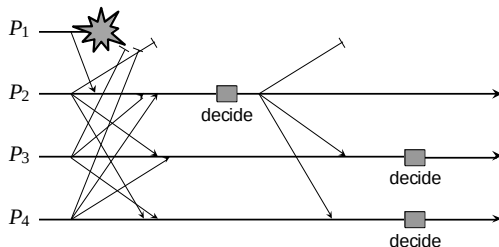
系统模型

- ▶ 进程组 $\mathbf{P} = \{P_1, \dots, P_n\}$
- ▶ **Fail-stop** 故障语义，即具有可靠的故障检测
- ▶ 客户端联系 P_i 请求其执行命令
- ▶ 每个 P_i 都维护一个建议命令列表

基本算法

1. 在轮次 r 中， P_i 将其已知的命令集 $\mathbf{C}^r$ 多播给所有其他命令集
2. 在 r 的末尾，每个 P_i 将所有收到的命令合并到新的 $\mathbf{C}^{r+1}$ 中。
3. 通过全局共享的确定性函数选择下一个命令 $cmd_i \leftarrow \text{select}(\mathbf{C}^{r+1})$.

Flooding-based consensus基于洪泛的共识：示例



观察

- ▶ P_2 收到所有其他进程提出的命令 $\Rightarrow$ 做出决策。
- ▶ P_3 可能检测到 P_1 崩溃，但不知道 P_2 是否收到任何消息，即 P_3 无法知道它是否具有与 P_2 相同的信息 $\Rightarrow$ 无法做出决策（ P_4 也是如此）。

Consistency, availability, and partitioning tolerance

一致性、可用性和分区容错性

CAP 定理

任何提供共享数据的网络系统只能提供以下三个属性中的两个：

C: consistency一致性, 共享和复制的数据项显示为单个最新副本

A: availability可用性, 更新最终总会被执行

P: Partioing tolerance分区容错性, 容忍进程组的分区。

结论

在容易发生通信故障的网络中，不可能实现保证对每个请求做出响应的原子读/写共享内存。

CAP 定理直觉

简单情况：两个相互作用的进程

- ▶ P 和 Q 不再能通信：
 - ▶ 允许 P 和 Q 继续 $\Rightarrow$ 不一致性
 - ▶ 仅允许 P、Q 中的一个继续 $\Rightarrow$ 不可用性
- ▶ 必须假设 P 和 Q 继续通信 $\Rightarrow$ 不允许分区。

Failure detection故障检测

问题

如何才能可靠地检测出某个进程是否真的崩溃了？

通用模型

- ▶ 每个进程都配备了一个故障检测模块
- ▶ 进程 P 探测另一个进程 Q 的反应
- ▶ 如果 Q 做出反应：Q 被认为是活着的（由 P 判断）
- ▶ 如果 Q 在 t 个时间单位内没有做出反应：Q 被怀疑崩溃了

对同步系统的观察

疑似崩溃 $\equiv$ 已崩溃

Practical failure detection实用故障检测

实现

- ▶ 如果 P 在时间 t 内没有收到 Q 的心跳：P 怀疑 Q。
- ▶ 如果 Q 稍后发送一条消息（P 收到）：
 - ▶ P 不再怀疑 Q
 - ▶ P 增加超时值 t
- ▶ 注意：如果 Q 确实崩溃了，P 将继续怀疑 Q。

可靠的远程过程调用

可能出现什么问题？

1. 客户端无法找到服务器。
2. 客户端向服务器发送的请求消息丢失。
3. 服务器在收到请求后崩溃。
4. 服务器向客户端发送的回复消息丢失。
5. 客户端在发送请求后崩溃。

可靠的远程过程调用

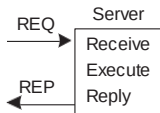
可能出现什么问题？

1. 客户端无法找到服务器。
2. 客户端向服务器发送的请求消息丢失。
3. 服务器在收到请求后崩溃。
4. 服务器向客户端发送的回复消息丢失。
5. 客户端在发送请求后崩溃。

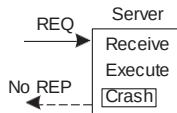
两个“简单”的解决方案

- 1: （无法找到服务器）：只需向客户端报告
- 2: （请求丢失）：只需重新发送消息

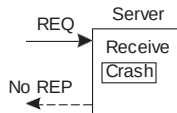
Reliable RPC: server crash



(a)



(b)



(c)

问题

其中 (a) 是正常情况，而情况 (b) 和 (c) 需要不同的解决方案。但是，我们不知道发生了什么。两种方法：

- ▶ 至少一次语义：无论如何，服务器保证它将至少执行一次操作。
- ▶ 最多一次语义：服务器保证它将最多执行一次操作。

可靠的 RPC：丢失回复消息

问题

客户端注意到的是，它没有得到答案。但是，它无法确定这是由丢失的请求、服务器崩溃还是丢失的响应引起的。

部分解决方法

设计服务器，使其操作具有幂等性：重复相同的操作与只执行一次相同：

- ▶ 纯读取操作
- ▶ 严格覆盖操作

许多操作本质上是非幂等的，例如许多银行交易。

可靠的 RPC : 客户端崩溃

问题

服务器正在工作并且无用地占有资源（称为进行孤立计算）。

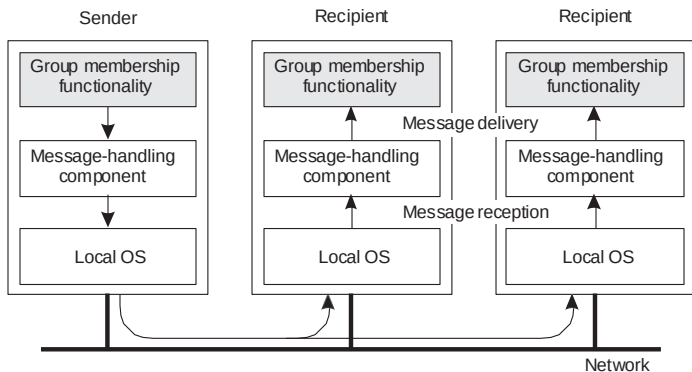
方法

- ▶ 客户端在恢复时会终止（或回滚）孤儿进程
- ▶ 客户端在恢复时广播新的纪元编号 $\Rightarrow$ 服务器终止客户端的孤儿进程
- ▶ 要求在 T 时间单位内完成计算。旧的计算将被删除。

Simple reliable group communication

Intuition

A message sent to a process group **G** should be delivered to each member of **G**. **Important**: make distinction between receiving and delivering messages.



Less simple reliable group communication

Reliable communication in the presence of faulty processes

Group communication is reliable when it can be guaranteed that a message is **received and subsequently delivered** by all **nonfaulty group members**.

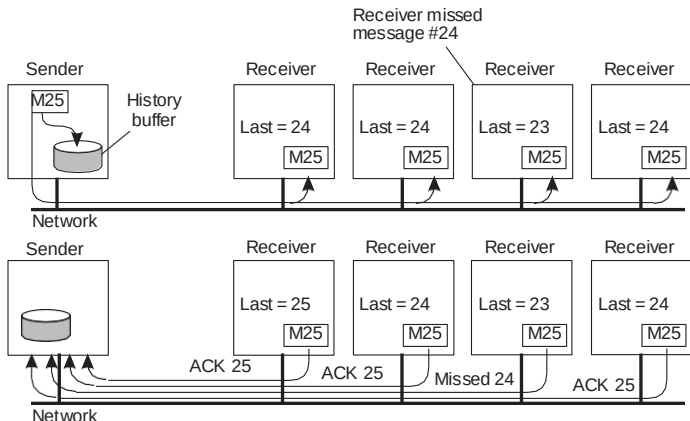
Tricky part

Agreement is needed on what the group actually looks like before a received message can be delivered.

Simple reliable group communication

Reliable communication, but assume nonfaulty processes

Reliable group communication now boils down to **reliable multicasting**: is a message received and delivered to each recipient, **as intended by the sender**.



Distributed commit protocols

Problem

Have an operation being performed by each member of a process group, or none at all.

- ▶ **Reliable multicasting**: a message is to be delivered to all recipients.
- ▶ **Distributed transaction**: each local transaction must succeed.

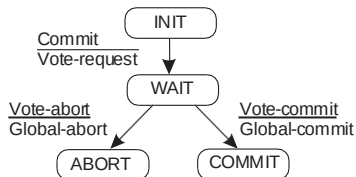
Two-phase commit protocol (2PC)

Essence

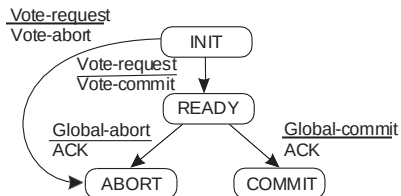
The client who initiated the computation acts as **coordinator**; processes required to commit are the **participants**.

- ▶ **Phase 1a:** Coordinator sends VOTE-REQUEST to participants (also called a **pre-write**)
- ▶ **Phase 1b:** When participant receives VOTE-REQUEST it returns either VOTE-COMMIT or VOTE-ABORT to coordinator. If it sends VOTE-ABORT, it aborts its local computation
- ▶ **Phase 2a:** Coordinator collects all votes; if all are VOTE-COMMIT, it sends GLOBAL-COMMIT to all participants, otherwise it sends GLOBAL-ABORT
- ▶ **Phase 2b:** Each participant waits for GLOBAL-COMMIT or GLOBAL-ABORT and handles accordingly.

2PC - Finite state machines



Coordinator



Participant

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- ▶ *INIT*: No problem: participant was unaware of protocol

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- ▶ **READY**: Participant is waiting to either commit or abort. After recovery, participant needs to know which state transition it should make $\Rightarrow$ log the coordinator's decision

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- ▶ **ABORT**: Merely make entry into abort state **idempotent**, e.g., removing the workspace of results

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- ▶ **COMMIT**: Also make entry into commit state idempotent, e.g., copying workspace to storage.

2PC – Failing participant

Analysis: participant crashes in state S , and recovers to S

- ▶ **INIT**: No problem: participant was unaware of protocol
- ▶ **READY**: Participant is waiting to either commit or abort. After recovery, participant needs to know which state transition it should make $\Rightarrow$ log the coordinator's decision
- ▶ **ABORT**: Merely make entry into abort state **idempotent**, e.g., removing the workspace of results
- ▶ **COMMIT**: Also make entry into commit state idempotent, e.g., copying workspace to storage.

Observation

When distributed commit is required, having participants use temporary workspaces to keep their results allows for simple recovery in the presence of failures.

2PC – Failing participant

Alternative

When a recovery is needed to *READY* state, check state of other participants $\Rightarrow$ no need to log coordinator's decision.

Recovering participant *P* contacts another participant *Q*

State of <i>Q</i>	Action by <i>P</i>
<i>COMMIT</i>	Make transition to <i>COMMIT</i>
<i>ABORT</i>	Make transition to <i>ABORT</i>
<i>INIT</i>	Make transition to <i>ABORT</i>
<i>READY</i>	Contact another participant

Result

If all participants are in the *READY* state, the protocol blocks. Apparently, the coordinator is failing. **Note:** The protocol prescribes that we need the decision from the coordinator.

2PC – Failing coordinator

Observation

The real problem lies in the fact that the coordinator's final decision may not be available for some time (or actually lost).

Alternative

Let a participant P in the *READY* state timeout when it hasn't received the coordinator's decision; P tries to find out what other participants know (as discussed).

Observation

Essence of the problem is that a recovering participant cannot make a **local** decision: it is dependent on other (possibly failed) processes

Coordinator in Python

```
1 class Coordinator:
2
3     def run(self):
4         yetToReceive = list(participants)
5         self.log.info('WAIT')
6         self.chan.sendTo(participants, VOTE_REQUEST)
7         while len(yetToReceive) > 0:
8             msg = self.chan.recvFrom(participants, TIMEOUT)
9             if (not msg) or (msg[1] == VOTE_ABORT):
10                 self.log.info('ABORT')
11                 self.chan.sendTo(participants, GLOBAL_ABORT)
12                 return
13             else: # msg[1] == VOTE_COMMIT
14                 yetToReceive.remove(msg[0])
15         self.log.info('COMMIT')
16         self.chan.sendTo(participants, GLOBAL_COMMIT)
```

Participant in Python

```
1 class Participant:
2     def run(self):
3         msg = self.chan.recvFrom(coordinator, TIMEOUT)
4         if (not msg): # Crashed coordinator - give up entirely
5             decision = LOCAL_ABORT
6         else: # Coordinator will have sent VOTE_REQUEST
7             decision = self.do_work()
8             if decision == LOCAL_ABORT:
9                 self.chan.sendTo(coordinator, VOTE_ABORT)
10            else: # Ready to commit, enter READY state
11                self.chan.sendTo(coordinator, VOTE_COMMIT)
12                msg = self.chan.recvFrom(coordinator, TIMEOUT)
13                if (not msg): # Crashed coordinator - check the others
14                    self.chan.sendTo(all_participants, NEED_DECISION)
15                while True:
16                    msg = self.chan.recvFromAny()
17                    if msg[1] in [GLOBAL_COMMIT, GLOBAL_ABORT, LOCAL_ABORT]:
18                        decision = msg[1]
19                        break
20                else: # Coordinator came to a decision
21                    decision = msg[1]
22        while True: # Help any other participant when coordinator crashed
23            msg = self.chan.recvFrom(all_participants)
24            if msg[1] == NEED_DECISION:
25                self.chan.sendTo([msg[0]], decision)
26
```

Recovery: Background

Essence

When a failure occurs, we need to bring the system into an error-free state:

- ▶ **Forward error recovery**: Find a new state from which the system can continue operation
- ▶ **Backward error recovery**: Bring the system back into a **previous** error-free state

Practice

Use backward error recovery, requiring that we establish **recovery points**

Observation

Recovery in distributed systems is complicated by the fact that processes need to cooperate in identifying a **consistent state** from where to recover

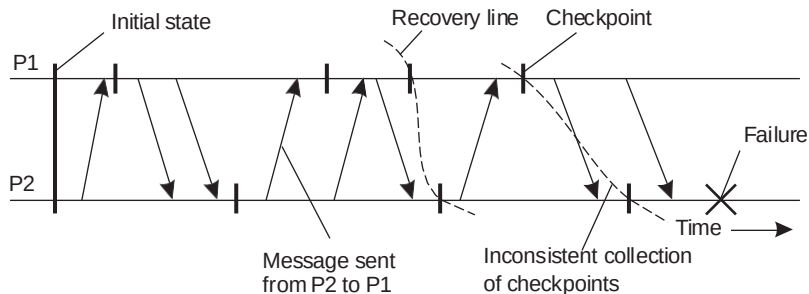
Consistent recovery state

Requirement

Every message that has been received is also shown to have been sent in the state of the sender.

Recovery line

Assuming processes regularly **checkpoint** their state, the most recent **consistent global checkpoint**.



Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

- ▶ A coordinator multicasts a **checkpoint request** message

Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

- ▶ A coordinator multicasts a **checkpoint request** message
- ▶ When a participant receives such a message, it takes a checkpoint, stops sending (application) messages, and reports back that it has taken a checkpoint

Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

- ▶ A coordinator multicasts a **checkpoint request** message
- ▶ When a participant receives such a message, it takes a checkpoint, stops sending (application) messages, and reports back that it has taken a checkpoint
- ▶ When all checkpoints have been confirmed at the coordinator, the latter broadcasts a **checkpoint done** message to allow all processes to continue

Coordinated checkpointing

Essence

Each process takes a checkpoint after a globally coordinated action.

Simple solution

Use a two-phase blocking protocol:

- ▶ A coordinator multicasts a **checkpoint request** message
- ▶ When a participant receives such a message, it takes a checkpoint, stops sending (application) messages, and reports back that it has taken a checkpoint
- ▶ When all checkpoints have been confirmed at the coordinator, the latter broadcasts a **checkpoint done** message to allow all processes to continue

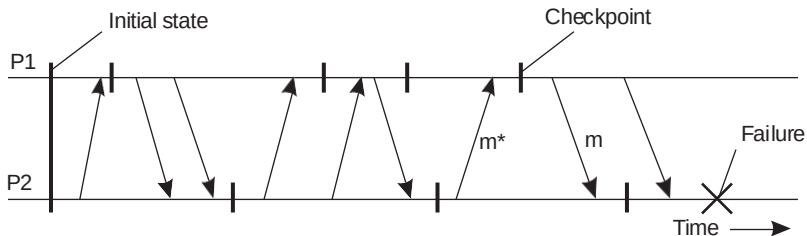
Observation

It is possible to consider only those processes that depend on the recovery of the coordinator, and ignore the rest

Cascaded rollback

Observation

If checkpointing is done at the “wrong” instants, the recovery line may lie at system startup time. We have a so-called **cascaded rollback**.



Independent checkpointing

Essence

Each process independently takes checkpoints, with the risk of a cascaded rollback to system startup.

Independent checkpointing

Essence

Each process independently takes checkpoints, with the risk of a cascaded rollback to system startup.

- ▶ Let $CP_i(m)$ denote m^{th} checkpoint of process P_i and $INT_i(m)$ the interval between $CP_i(m-1)$ and $CP_i(m)$.

Independent checkpointing

Essence

Each process independently takes checkpoints, with the risk of a cascaded rollback to system startup.

- ▶ Let $CP_i(m)$ denote m^{th} checkpoint of process P_i and $INT_i(m)$ the interval between $CP_i(m-1)$ and $CP_i(m)$.
- ▶ When process P_i sends a message in interval $INT_i(m)$, it piggybacks (i, m)

Independent checkpointing

Essence

Each process independently takes checkpoints, with the risk of a cascaded rollback to system startup.

- ▶ Let $CP_i(m)$ denote m^{th} checkpoint of process P_i and $INT_i(m)$ the interval between $CP_i(m-1)$ and $CP_i(m)$.
- ▶ When process P_i sends a message in interval $INT_i(m)$, it piggybacks (i, m)
- ▶ When process P_j receives a message in interval $INT_j(n)$, it records the dependency $INT_i(m) \rightarrow INT_j(n)$.

Independent checkpointing

Essence

Each process independently takes checkpoints, with the risk of a cascaded rollback to system startup.

- ▶ Let $CP_i(m)$ denote m^{th} checkpoint of process P_i and $INT_i(m)$ the interval between $CP_i(m-1)$ and $CP_i(m)$.
- ▶ When process P_i sends a message in interval $INT_i(m)$, it piggybacks (i, m)
- ▶ When process P_j receives a message in interval $INT_j(n)$, it records the dependency $INT_i(m) \rightarrow INT_j(n)$.
- ▶ The dependency $INT_i(m) \rightarrow INT_j(n)$ is saved to storage when taking checkpoint $CP_j(n)$.

Independent checkpointing

Essence

Each process independently takes checkpoints, with the risk of a cascaded rollback to system startup.

- ▶ Let $CP_i(m)$ denote m^{th} checkpoint of process P_i and $INT_i(m)$ the interval between $CP_i(m-1)$ and $CP_i(m)$.
- ▶ When process P_i sends a message in interval $INT_i(m)$, it piggybacks (i, m)
- ▶ When process P_j receives a message in interval $INT_j(n)$, it records the dependency $INT_i(m) \rightarrow INT_j(n)$.
- ▶ The dependency $INT_i(m) \rightarrow INT_j(n)$ is saved to storage when taking checkpoint $CP_j(n)$.

Observation

If process P_i rolls back to $CP_i(m-1)$, P_j must roll back to $CP_j(n-1)$.

Message logging

Alternative

Instead of taking an (expensive) checkpoint, try to **replay** your (communication) behavior from the most recent checkpoint $\Rightarrow$ store messages in a log.

Assumption

We assume a **piecewise deterministic** execution model:

- ▶ The execution of each process can be considered as a sequence of state intervals
- ▶ Each state interval starts with a nondeterministic event (e.g., message receipt)
- ▶ Execution in a state interval is deterministic

Conclusion

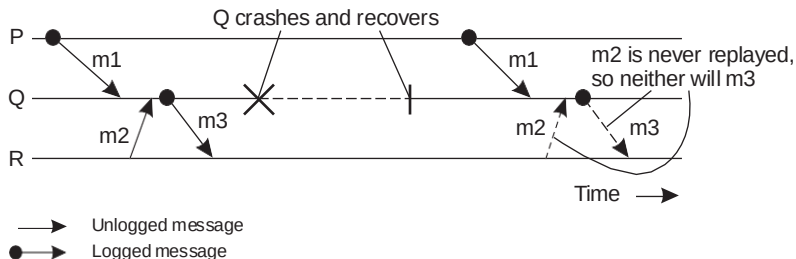
If we record nondeterministic events (to replay them later), we obtain a deterministic execution model that will allow us to do a complete replay.

Message logging and consistency

When should we actually log messages?

Avoid **orphan processes**:

- ▶ Process Q has just received and delivered messages m_1 and m_2
- ▶ Assume that m_2 is never logged.
- ▶ After delivering m_1 and m_2 , Q sends message m_3 to process R
- ▶ Process R receives and subsequently delivers m_3 : it is an orphan.



Message-logging schemes

Notations

- ▶ **DEP**(m): processes to which m has been delivered. If message m^* is causally dependent on the delivery of m , and m^* has been delivered to Q , then $Q \in \mathbf{DEP}(m)$.
- ▶ **COPY**(m): processes that have a copy of m , but have not (yet) reliably stored it.
- ▶ **FAIL**: the collection of crashed processes.

Characterization

Q is orphaned $\Leftrightarrow \exists m. Q \in \mathbf{DEP}(m)$ and $\mathbf{COPY}(m) \subseteq \mathbf{FAIL}$

Message-logging schemes

Pessimistic protocol

For each **nonstable** message m , there is at most one process dependent on m , that is $|\mathbf{DEP}(m)| \leq 1$.

Consequence

An unstable message in a pessimistic protocol **must** be made stable before sending a next message.

Message-logging schemes

Optimistic protocol

For each unstable message m , we ensure that if **COPY**(m) $\subseteq$ **FAIL**, then eventually also **DEP**(m) $\subseteq$ **FAIL**.

Consequence

To guarantee that **DEP**(m) $\subseteq$ **FAIL**, we generally rollback each orphan process Q until $Q \notin \mathbf{DEP}(m)$.